Fraud Risk Ranking Engine

Results report | Roll number 2013 | Section A

Problem A: know the data

The file has 284,807 transactions and 492 frauds. There are no missing, nonnumeric, or non-finite cells. There are 1,081 duplicate full rows; they were kept. Amount has 32,767 distinct values, and the ten most frequent values account for 16.29% of rows. Time is nondecreasing in the raw file. The raw file is neither triage sorted nor its exact reverse. C2 to C6 are constructed conditions; the raw file is C1.

Band	Transactions	Share	Frauds	Fraud share
0	0	0.00%	0	0.00%
1	0	0.00%	0	0.00%
2	21,156	7.43%	0	0.00%
3	107,410	37.71%	13	2.64%
4	113,116	39.72%	30	6.10%
5	32,084	11.27%	23	4.67%
6	6,006	2.11%	56	11.38%
7	2,797	0.98%	97	19.72%
8	1,376	0.48%	112	22.76%
9	862	0.30%	161	32.72%

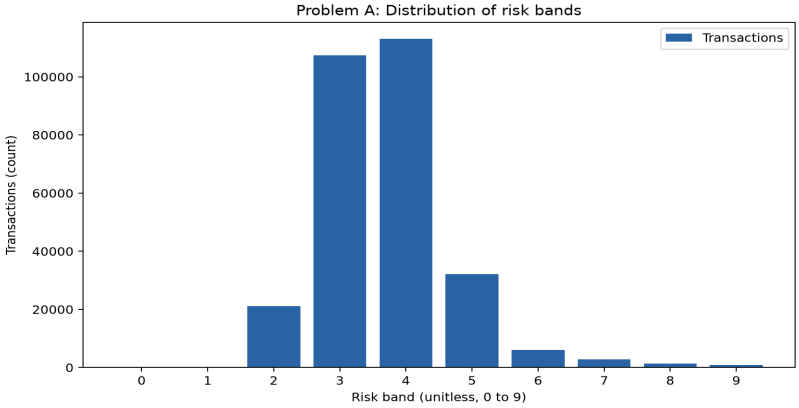


Figure A1. Full file risk band distribution. Bands use population standard deviation (ddof=0).

Problem B: implementation

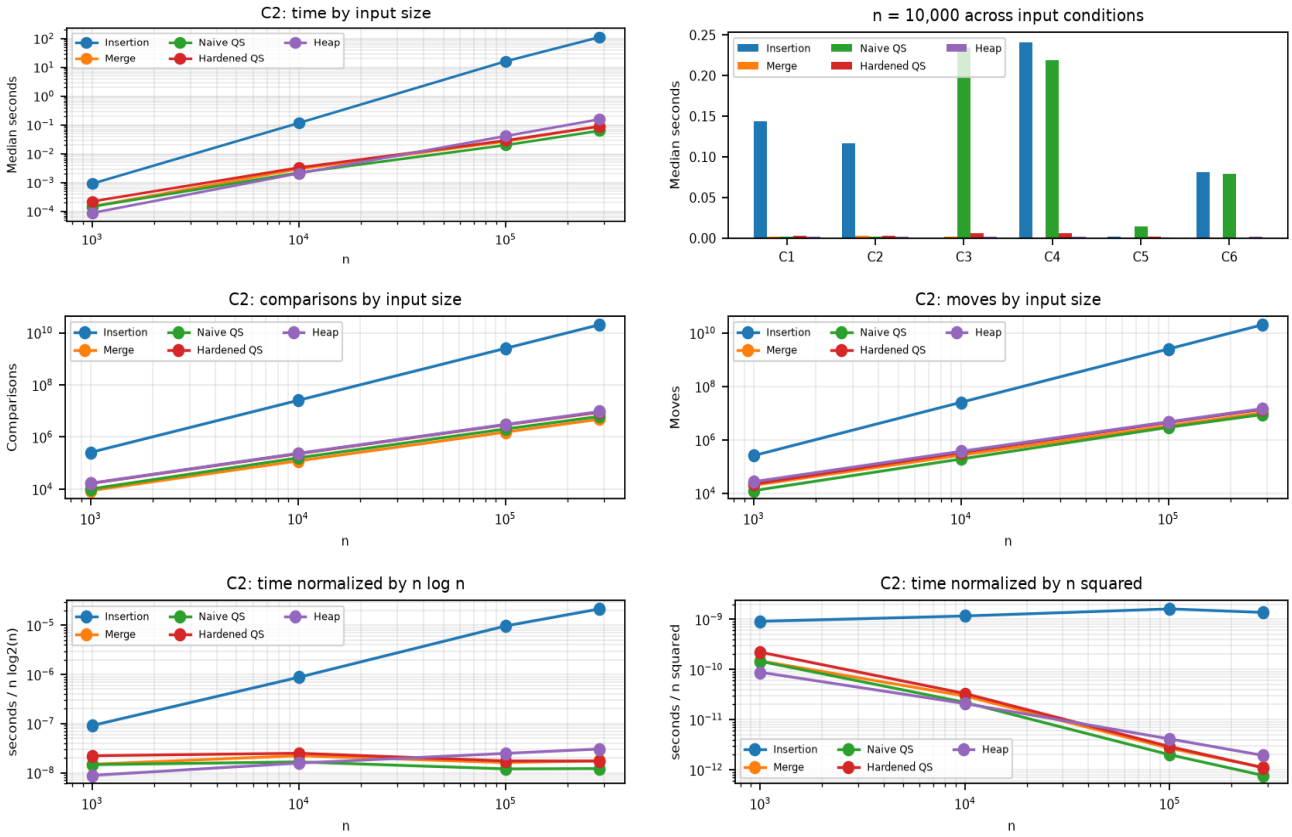
The C++17 implementation contains five full-sort algorithms, two top-K methods, three live-feed strategies, comparator counters, and correctness checks. Tests cover empty, singleton, duplicates, equivalent keys, and preservation of records.

Experiment setup: five trials, median summaries, seed 2013, personal K=230. C and D use the five minute limit. All C and D trials completed. `Class` is used only for fraud-capture evaluation.

Problem C: full ranking tournament

There are 24 prediction rounds. Winner predictions were correct in 10/24; loser predictions were correct in 18/24. The full log is included in the appendix and `prediction_log.csv`.

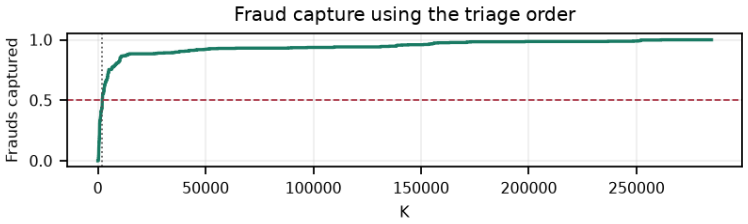
n	Insertion	Merge	Naive QS	Hardened QS	Heap
1,000	0.0009088	0.0001481	0.0001452	0.0002209	8.79e-05
10,000	0.1163	0.00295	0.002203	0.003303	0.002085
100,000	16.09	0.0267	0.01987	0.02886	0.04106
284,807	111.1	0.09056	0.06255	0.08906	0.1568



C2's fastest method was Naive Quick Sort at n=100,000 and 284,807. Across all conditions the winner varied: insertion sort excelled on exact triage order, while Merge Sort led on the reverse input at full size. All requested plots are also available as SVG files beside the CSV results.

Problems D and E: selective and live queues

D compares full Merge Sort, the size-K min heap, and the bounded insertion buffer across C1/C2 and both sample sizes. K includes 10, 50, 230, 500, 5,000, n/10, plus 2,068 for the half-fraud threshold. The size-K heap was fastest at every tested point. At n=284,807, bounded-buffer time rose from 5.28 to 5.37 seconds at K=2,068, 6.15 to 9.47 seconds at K=5,000, and 56.67 to 72.82 seconds at K=28,480.

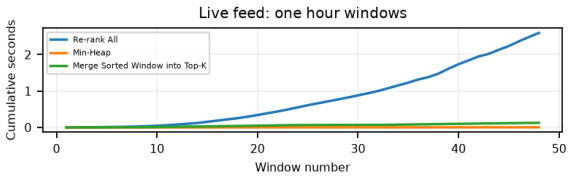


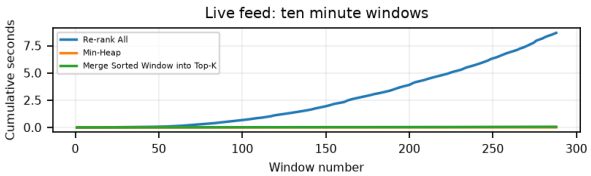
Condition	Method	Seconds	Comparisons	Moves	Bytes
C1	Merge Sort	0.1872	4,792,310	10,343,704	9,113,824
C1	Size-K Min-Heap	0.008738	527,362	367,438	66,176
C1	Bounded Insertion Buffer	5.276	574,512,669	12,345,601	66,176
C2	Merge Sort	0.195	4,804,507	10,343,704	9,113,824
C2	Size-K Min-Heap	0.007042	518,861	354,179	66,176
C2	Bounded Insertion Buffer	5.374	575,157,234	11,700,656	66,176

The smallest K capturing at least half the frauds is 2,068 of 492. Full-sort, heap, and buffer median costs for this K are shown above; `half_capture_cost.csv` has exact values.

For E, the heap fed only by newly arrived transactions was fastest and used the least extra memory for both window sizes. The window inputs are split by Time; memory estimates exclude the preloaded dataset and input windows.

Window	Strategy	Final sec	Peak bytes
1-hour	Min-Heap	0.004535	14,720
1-hour	Merge Sorted Window into Top-K	0.1265	1,055,936
1-hour	Re-rank Merge Sort	2.598	35,006,336
10-minute	Min-Heap	0.008049	14,720
10-minute	Merge Sorted Window into Top-K	0.06587	269,504
10-minute	Re-rank Merge Sort	8.668	20,364,160





Complexity and recommendations

Method	Time	Extra storage	Stable
Insertion Sort	O(n squared)	O(1)	Yes
Merge Sort	O(n log n)	O(n)	Yes
Naive Quick Sort	Average O(n log n), worst O(n squared)	O(log n) average; O(n) worst stack	No
Hardened Quick Sort	Expected O(n log n), worst O(n squared)	O(log n) average; O(n) worst stack	No
Heap Sort	O(n log n)	O(1)	No
Size-K min heap	O(n log K)	O(K)	No
Bounded insertion buffer	O(nK) worst case	O(K)	Yes

Use Merge Sort for predictable full ranking, a size-K min heap when only K records are needed, and a heap fed by each new window for the live feed. These choices match the measured performance and memory results.

Scope and source notes

The separately assigned F and H material was reviewed against these refreshed measurements. Problem G is omitted from this project work as requested. The optional local Analyst Console lets an analyst select a queue algorithm and K, view ranked transaction fields, and inspect runtime and comparisons. Its bonus remains conditional on complete A to H assessment.

AI use disclosure: AI assistance was used to inspect and edit code, organize experiment outputs, and prepare documentation. The student should review the source, measurements, and report before submission and can explain the implementation.

Prediction log appendix

Each prediction was recorded before its tournament round. The result rows below show the predicted and actual winner and loser. Complete reasoning text is preserved in `prediction_log.csv`.

Round	n	Condition	Pred winner	Actual winner	Pred loser	Actual loser	Reason
C1	1,000	C1	Merge Sort	Heap Sort	Insertion Sort	Insertion Sort	The arrival order has no known relation to the complete triage key. Merge Sort keeps n log n work.
C2	1,000	C2	Hardened Quick Sort	Heap Sort	Insertion Sort	Insertion Sort	The shuffle removes existing order; three way partitioning protects Quick Sort and insertion can shift many records.
C3	1,000	C3	Insertion Sort	Insertion Sort	Naive Quick Sort	Naive Quick Sort	A ranked input should need few insertion shifts, while the last pivot on sorted data creates unbalanced partitions.
C4	1,000	C4	Heap Sort	Heap Sort	Naive Quick Sort	Naive Quick Sort	Heap Sort keeps n log n work; reverse order is a worst case pattern for the last pivot.
C5	1,000	C5	Insertion Sort	Insertion Sort	Naive Quick Sort	Naive Quick Sort	Only one percent of records were moved, so insertion shifts should stay limited; the last pivot remains sensitive to long ordered runs.
C6	1,000	C6	Hardened Quick Sort	Hardened Quick Sort	Naive Quick Sort	Naive Quick Sort	Band-only keys have many ties; three way partition handles equal keys while Lomuto can repeatedly split off one item.
C7	10,000	C1	Merge Sort	Naive Quick Sort	Insertion Sort	Insertion Sort	The arrival order has no known relation to the complete triage key. Merge Sort keeps n log n work.
C8	10,000	C2	Hardened Quick Sort	Heap Sort	Insertion Sort	Insertion Sort	The shuffle removes existing order; three way partitioning protects Quick Sort and insertion can shift many records.
C9	10,000	C3	Insertion Sort	Insertion Sort	Naive Quick Sort	Naive Quick Sort	A ranked input should need few insertion shifts, while the last pivot on sorted data creates unbalanced partitions.
C10	10,000	C4	Heap Sort	Merge Sort	Naive Quick Sort	Insertion Sort	Heap Sort keeps n log n work; reverse order is a worst case pattern for the last pivot.
C11	10,000	C5	Insertion Sort	Merge Sort	Naive Quick Sort	Naive Quick Sort	Only one percent of records were moved, so insertion shifts should stay limited; the last pivot remains sensitive to long ordered runs.
C12	10,000	C6	Hardened Quick Sort	Hardened Quick Sort	Naive Quick Sort	Insertion Sort	Band-only keys have many ties; three way partition handles equal keys while Lomuto can repeatedly split off one item.
C13	100,000	C1	Merge Sort	Naive Quick Sort	Insertion Sort	Insertion Sort	The arrival order has no known relation to the complete triage key. Merge Sort keeps n log n work.
C14	100,000	C2	Hardened Quick Sort	Naive Quick Sort	Insertion Sort	Insertion Sort	The shuffle removes existing order; three way partitioning protects Quick Sort and insertion can shift many records.
C15	100,000	C3	Insertion Sort	Insertion Sort	Naive Quick Sort	Naive Quick Sort	A ranked input should need few insertion shifts, while the last pivot on sorted data creates unbalanced partitions.
C16	100,000	C4	Heap Sort	Merge Sort	Naive Quick Sort	Insertion Sort	Heap Sort keeps n log n work; reverse order is a worst case pattern for the last pivot.
C17	100,000	C5	Insertion Sort	Merge Sort	Naive Quick Sort	Naive Quick Sort	Only one percent of records were moved, so insertion shifts should stay limited; the last pivot remains sensitive to long ordered runs.
C18	100,000	C6	Hardened Quick Sort	Hardened Quick Sort	Naive Quick Sort	Insertion Sort	Band-only keys have many ties; three way partition handles equal keys while Lomuto can repeatedly split off one item.
C19	284,807	C1	Merge Sort	Naive Quick Sort	Insertion Sort	Insertion Sort	The arrival order has no known relation to the complete triage key. Merge Sort keeps n log n work.
C20	284,807	C2	Hardened Quick Sort	Naive Quick Sort	Insertion Sort	Insertion Sort	The shuffle removes existing order; three way partitioning protects Quick Sort and insertion can shift many records.
C21	284,807	C3	Insertion Sort	Insertion Sort	Naive Quick Sort	Naive Quick Sort	A ranked input should need few insertion shifts, while the last pivot on sorted data creates unbalanced partitions.
C22	284,807	C4	Heap Sort	Merge Sort	Naive Quick Sort	Insertion Sort	Heap Sort keeps n log n work; reverse order is a worst case pattern for the last pivot.
C23	284,807	C5	Insertion Sort	Merge Sort	Naive Quick Sort	Insertion Sort	Only one percent of records were moved, so insertion shifts should stay limited; the last pivot remains sensitive to long ordered runs.
C24	284,807	C6	Hardened Quick Sort	Hardened Quick Sort	Naive Quick Sort	Naive Quick Sort	Band-only keys have many ties; three way partition handles equal keys while Lomuto can repeatedly split off one item.

Prediction score: 10 of 24 winners and 18 of 24 losers predicted correctly. Each round had a written reason in the source log.